

■ Design Amazon / E-Commerce Platform

System Design Interview | E-Commerce & Marketplace

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Product catalog; Search; Cart; Checkout; Order management; Inventory tracking; Recommendations; Reviews & ratings
Non-Functional	High availability (99.99%); Peak traffic: Prime Day 10× normal; <200ms page load; Accurate inventory to prevent overselling
Scale	300M+ users; 350M+ products; 1.6M orders/day; Inventory updates must be strongly consistent to avoid oversell

■ High-Level Architecture (Microservices)

- Product Service (catalog, search), Inventory Service, Cart Service, Order Service, Payment Service, Notification Service — each independently deployable
- API Gateway routes traffic; each service has its own DB (database-per-service pattern); async communication via Kafka
- Product search: Elasticsearch index on product title, category, brand, price; faceted search with filters
- CDN serves product images/static assets; origin is S3; 95%+ cache hit on product images

■ Cart Service Design

- **Storage:** Redis for active carts (fast, TTL 30 days); persist to DB on checkout or session expiry; `cart_id` → `{items: [{product_id, qty, price_at_add}]}`
- **Price Lock:** Store `price_at_add` in cart; show price change warning if current price differs at checkout
- **Guest Cart:** Anonymous cart in Redis with cookie-based `cart_id`; merge with user cart on login
- **Concurrency:** Redis MULTI/EXEC (optimistic locking) for concurrent cart updates from multiple devices; last-write-wins per item

■ Inventory Management (Critical Path)

- Inventory DB: PostgreSQL with row-level locking; `inventory(product_id PK, qty_available, qty_reserved)`
- Reserve on add-to-cart (soft reserve, TTL 15 min); deduct on successful payment; release on timeout/cancel
- Pessimistic locking: `SELECT ... FOR UPDATE` on inventory row during checkout → prevents concurrent oversell
- Distributed inventory: each warehouse has local count; aggregate for display; deduct from nearest warehouse on order
- Flash sale: pre-reduce inventory to Redis counter (DECR); atomic and fast; sync back to DB post-sale

■ Order Management State Machine

- States: `PENDING` → `PAYMENT_PROCESSING` → `CONFIRMED` → `WAREHOUSE_PICKED` → `SHIPPED` → `DELIVERED` → `(RETURN_REQUESTED → RETURNED)`

- Each state transition persisted to DB; events published to Kafka for downstream services (notification, analytics)
- Idempotency: order_id as idempotency key; retry-safe checkout API; exactly-once payment processing
- Order history: Cassandra (partition by user_id, cluster by created_at DESC) for fast user order timeline queries

■■ Inventory Consistency Trade-offs

Pessimistic Locking (SQL)	Optimistic + Redis
✓ No oversell guaranteed	✓ High throughput flash sales
✓ Slower under contention	✓ DECR in Redis is atomic
✓ Simpler logic	✓ Sync to DB async
✓ Good for low-concurrency SKUs	✓ Slight oversell risk on failure

■ Product Search & Recommendations

- Elasticsearch: inverted index on title/description/brand; BM25 relevance scoring; filtered aggregations for facets (category, price range, rating)
- Autocomplete: edge n-gram tokenizer in ES or separate Redis-based trie for sub-50ms suggestions
- Recommendations: collaborative filtering (users who bought X also bought Y); item2vec embeddings; real-time via Kafka user-action stream
- A/B testing: route % of traffic to ranking variants; measure CTR and conversion per variant

■ Checkout Flow (Distributed Transaction)

- Saga pattern: Reserve Inventory → Charge Payment → Confirm Order → Send Notification; each step has compensating rollback
- If payment fails: release inventory reservation; notify user; log failure for retry
- Idempotency keys per step prevent double charges on network retries
- Two-phase commit avoided in favor of Saga for higher availability across services